

Módulo 5 – Capítulo 09 – Sección 01

Anatomía del costo: tokens de entrada, salida, caché y llamadas a herramientas

El gasto en APIs de LLM es uno de los pocos costos de infraestructura en tecnología que puede variar en un orden de magnitud entre una implementación eficiente y una ineficiente con el mismo nivel de funcionalidad: el mismo sistema de chat puede costar \$1.200 o \$8.000 al mes con 100.000 requests diarios, dependiendo de si el equipo eligió el modelo correcto para cada tarea, implementó prompt caching, controla la longitud del historial conversacional, y usa batching donde la latencia lo permite. Entender la anatomía del costo — de qué partes se compone la factura de cada request— es el prerequisite para cualquier optimización que no sea aleatoria.

La factura de una llamada al LLM se compone de cuatro componentes con precios radicalmente diferentes. Los tokens de entrada —el prompt completo incluyendo system prompt, historial conversacional, documentos recuperados y la query del usuario— representan el mayor volumen de tokens en la mayoría de los sistemas pero el menor precio por token: \$3 por millón en Claude 3.5 Sonnet. Los tokens de salida —la respuesta generada— son el componente de mayor precio por token: \$15 por millón en el mismo modelo, cinco veces más que los de entrada. Esta asimetría tiene una implicación directa de diseño: reducir la longitud de las respuestas del LLM —instruyendo al modelo a ser conciso, usando formatos compactos como JSON en lugar de prosa extendida cuando los datos son lo que importa— reduce el costo de forma más efectiva que reducir el tamaño del prompt de entrada.

Los tokens de caché de Anthropic introducen una tercera categoría con precio diferencial. Los tokens de cache write —cuando se escribe un nuevo prefijo al caché— cuestan \$3.75 por millón, un 25% más caro que la lectura normal. Los tokens de cache read —cuando el mismo prefijo se reutiliza en llamadas subsiguientes— cuestan \$0.30 por millón, un 90% de descuento sobre el precio normal. El breakeven se alcanza con apenas 1.25 llamadas que reutilicen el mismo prefijo: después de esa primera llamada, cada reutilización ahorra el 90% del costo del prefijo cacheado. Para un system prompt de 3.000 tokens en Claude 3.5 Sonnet,

el ahorro por cada cache hit es de \$0.0081 —pequeño individualmente pero significativo a 100.000 requests diarios: \$810 diarios o \$29.000 mensuales de ahorro potencial.

Los costos multimodales y de herramientas añaden complejidad adicional. Las imágenes en OpenAI se facturan por "tiles" de 512×512 píxeles: una imagen de 1024×1024 ocupa 4 tiles y tiene un costo equivalente a varios centenares de tokens de texto. Los tool use blocks en el array de mensajes de conversaciones multi-turno con herramientas se cuentan como tokens de entrada en cada turno subsiguiente, acumulando el historial de herramientas en el contexto.

Componentes del costo de las APIs de LLM

- **Tokens de entrada:** incluyen system prompt, historial conversacional, documentos recuperados por RAG, ejemplos few-shot, y la query del usuario; crecen sin control si no se implementan límites de contexto explícitos.
- **Tokens de salida:** menor volumen pero mayor precio (3-5x más que entrada); controlables con `max_tokens`, con instrucciones de concisión en el prompt, y eligiendo formatos compactos (JSON vs narrativa extendida).
- **Cache write tokens:** 25% más caro que lectura normal en Anthropic; costo de escritura amortizado en 1.25 llamadas con el mismo prefijo; activable marcando bloques con `cache_control: {"type": "ephemeral"}`.
- **Cache read tokens:** 90% de descuento sobre el precio normal; el mecanismo de mayor impacto en costo para sistemas con system prompts largos o documentos de referencia fijos por sesión.
- **Costos multimodales:** imágenes facturadas por tiles de 512×512px; documentos PDF en Anthropic facturados por página como imagen; tool use blocks acumulados en el historial de mensajes como tokens de entrada en turnos subsiguientes.

La instrumentación del costo en cada request —campo `cost_usd` calculado localmente y loggeado— es el prerequisite para cualquier optimización. Sin saber cuánto cuesta cada request desagregado por componente y por feature, la optimización de costos es ciega. Con instrumentación completa, las oportunidades de ahorro más grandes son evidentes desde el primer análisis de los datos de producción.

Principio rector: El costo total de un sistema de IA es predecible e instrumentable; la variabilidad de costo sin instrumentación convierte el presupuesto en una sorpresa mensual

en lugar de una métrica gestionada activamente por el equipo de ingeniería.

Módulo 5 – Capítulo 09 – Sección 02

Estrategias de reducción: compresión de contexto, caché semántica y batching

Las estrategias de reducción de costo en sistemas de IA atacan el problema desde tres ángulos complementarios: reducir la cantidad de tokens enviados en cada llamada (compresión de contexto), reutilizar respuestas de llamadas anteriores cuando la pregunta es suficientemente similar (caché semántica), y consolidar múltiples peticiones en una sola llamada cuando el caso de uso lo permite (batching). La regla de oro que debe preceder a cualquier implementación es: medir el impacto de cada estrategia en la calidad del sistema con el dataset de evaluación antes de desplegarla en producción, porque cada una de estas optimizaciones implica un trade-off potencial con la calidad que solo el sistema de evaluación puede cuantificar.

La compresión de contexto en sistemas conversacionales es la optimización de mayor retorno para sistemas con sesiones largas porque los tokens del historial son el componente que más crece con el tiempo. La estrategia de sliding window —mantener los últimos K turnos completos— es la más simple: `messages = messages[-2*K:]` donde K es el número de pares usuario-asistente a conservar, configurable por el requisito de coherencia del dominio. La estrategia de summarization es más costosa —requiere una llamada adicional al LLM con `temperature=0` para generar el resumen— pero preserva información de largo plazo mejor que la sliding window pura: `summary = llm.summarize(messages[:-K]); messages = [{"role": "system", "content": f"Historial resumido: {summary}"}] + messages[-K:]`. El costo del resumen —una llamada de ~1.000 tokens de entrada y ~500 de salida, aproximadamente \$0.012 en Claude 3.5 Sonnet— se amortiza en todos los turnos futuros que usan el historial comprimido en lugar del historial completo.

La caché semántica extiende el concepto de caching más allá de la igualdad exacta de texto: en lugar de cachear solo respuestas a preguntas literalmente idénticas, cachea respuestas a preguntas semánticamente equivalentes. El proceso tiene dos pasos: generar el embedding de la query del usuario con un modelo eficiente (`text-embedding-3-small` de OpenAI o `embed-multilingual-v3.0` de Cohere), buscar en Redis o en Qdrant por similitud coseno con las

preguntas cacheadas, y devolver la respuesta cacheada si la similitud supera el umbral configurado (típicamente 0.95). El TTL del caché debe alinearse con la frecuencia de cambio de los datos del dominio: para una base de conocimiento de productos que se actualiza mensualmente, un TTL de 24 horas es apropiado; para preguntas sobre datos en tiempo real, la caché semántica no aplica. En dominios con muchas preguntas frecuentes repetitivas — FAQs, consultas sobre catálogos de productos, preguntas sobre políticas de empresa—, el hit rate puede superar el 40%, con ahorro proporcional.

El batching consolida múltiples consultas independientes en una sola llamada al LLM cuando la latencia por item individual no es el requisito principal. En lugar de N llamadas separadas para clasificar N documentos con el mismo prompt, una sola llamada con los N documentos separados por delimitadores XML (`<document id="1">...</document><document id="2">...</document>`) reduce el costo del system prompt repetido N veces a una única instancia. Para pipelines de procesamiento nocturno o de enriquecimiento de datos, el Batch API de Anthropic permite enviar hasta 10.000 requests en un lote con hasta un 50% de descuento sobre el precio normal, a cambio de un SLA de latencia de hasta 24 horas.

Estrategias técnicas de reducción de costos

- **Compresión de historial con sliding window:** `messages = messages[-2*K:]` ; valor de K configurable en `config/conversation.yaml` ; 10 a 20 turnos son suficientes para la coherencia en la mayoría de los casos de uso; monitor del costo por turno antes y después para cuantificar el ahorro.
- **Caché semántica con embeddings y Redis:** embedding de la query con `text-embedding-3-small` , búsqueda por similitud coseno con `redis.FT.SEARCH` o en Qdrant; threshold de 0.95 para evitar servir respuestas de preguntas superficialmente similares pero semánticamente diferentes.
- **Batching de requests independientes:** un único prompt con N documentos separados por `<doc id="N">` reduce el overhead del system prompt de N copias a una; especialmente efectivo para clasificación y extracción de entidades en batch.
- **Truncación selectiva del contexto RAG:** de 5 chunks a 3 chunks de 500 tokens cuando el sistema de evaluación confirma que el tercer chunk raramente contribuye a la respuesta; ahorro de 1.000 tokens de entrada por request verificado con métricas de context recall.
- **Reutilización de prefix con prompt caching del proveedor:** en sistemas con system prompt largo (>1.000 tokens) y documentos de referencia fijos, marcar con

`cache_control` para que las llamadas sucesivas en la misma sesión paguen solo el costo de cache read; especialmente impactante en sesiones multi-turno.

Medir el impacto de cada estrategia de reducción de costo con el sistema de observabilidad antes y después de implementarla es la única forma de confirmar que el ahorro no viene acompañado de degradación de calidad que supere el valor del ahorro económico.

Buena práctica: Medir el impacto en calidad de cada estrategia de reducción de costo con el dataset de evaluación antes de implementarla en producción; la compresión de contexto reduce costo pero puede degradar la coherencia del diálogo si el resumen pierde información crítica del historial; solo la medición confirma que el trade-off es aceptable para el caso de uso específico.

Módulo 5 – Capítulo 09 – Sección 03

Prompt caching: Anthropic, OpenAI y su impacto en costos repetitivos

El prompt caching es la funcionalidad de proveedor con el mayor ratio de impacto en costo versus complejidad de implementación disponible en el ecosistema de LLMs: el cambio necesario en el código es agregar un campo `cache_control` en el cuerpo del request de Anthropic, o simplemente estructurar el prompt con prefijos largos en OpenAI donde el caching es automático. El ahorro resultante puede llegar al 80-90% en el costo de los tokens de entrada repetitivos. Pocos cambios en un sistema de IA tienen un impacto de costo tan alto con tan poca complejidad de implementación.

La implementación en Anthropic es explícita: el desarrollador marca qué bloques del prompt deben cachearse con `cache_control: {"type": "ephemeral"}`. El bloque marcado —el system prompt, un documento de referencia, una lista de ejemplos few-shot— se cachea en los servidores de Anthropic durante cinco minutos desde el último uso, con renovación automática del TTL en cada llamada que reutiliza el mismo prefijo. La condición de éxito es que el contenido marcado para caché sea textualmente idéntico entre llamadas —cualquier diferencia en un solo carácter invalida el cache hit— y que supere el mínimo de 1.024 tokens para activarse. La estructura óptima de prompt para maximizar el cache hit rate es "static first, dynamic last": `[system prompt estático - 2.000 tokens][documentos de referencia fijos - 5.000 tokens]` marcados con `cache_control`, seguido de `[historial conversacional reciente][query variable del usuario]` sin marcar.

La implementación en OpenAI es automática para prefijos de 1.024 tokens o más: el proveedor cachea automáticamente los prefijos de los prompts sin requerir configuración explícita, aplicando un descuento del 50% sobre el precio de entrada para los tokens cacheados. El campo `usage.prompt_tokens_details.cached_tokens` en la respuesta indica cuántos tokens vinieron del caché para cada llamada. La activación automática tiene la ventaja de no requerir ningún cambio de código, pero la desventaja de no proveer control explícito sobre qué se cachea: el caching ocurre cuando la infraestructura de OpenAI lo determina, que puede no ser consistente en todos los requests.

Google Gemini Context Caching tiene una API separada con características diferentes:

```
genai.caching.CachedContent.create(contents=..., ttl=datetime.timedelta(hours=1))
```

crea un objeto de caché en la nube con un TTL configurable y un costo de almacenamiento por token por hora (\$1 por millón de tokens por hora). Este modelo es diferente al de Anthropic: el caché tiene un costo de mantenimiento por tiempo en lugar de un costo de escritura único, y el beneficio se materializa cuando el volumen de llamadas que reutilizan el mismo contenido es suficientemente alto para justificar el costo de almacenamiento. El mínimo de 4.096 tokens para activar Context Caching en Gemini limita su aplicabilidad a contextos suficientemente grandes.

Aspectos técnicos del prompt caching

- **Estructura óptima para caching en Anthropic:** `[system prompt estático]` `[documentos fijos]` con `cache_control: {"type": "ephemeral"}` seguido de `[query variable]`; el prefijo estático debe ser textualmente idéntico entre llamadas para activar el cache hit; cualquier diferencia invalida el caché.
- **Monitoreo del cache hit rate:** campo `usage.cache_read_input_tokens` en la respuesta de Anthropic indica cuántos tokens vinieron del caché; monitorizar este campo como métrica operativa con alerta cuando cae por debajo del expected para detectar cambios en la estructura del prompt que reducen el hit rate.
- **Casos de uso de mayor ahorro:** system prompts largos con instrucciones de empresa (>1.000 tokens), documentos de referencia fijos por sesión (contratos, manuales de producto), y ejemplos few-shot estáticos que se repiten en cada llamada.
- **TTL y renovación en Anthropic:** el caché dura 5 minutos desde el último uso; para sesiones conversacionales activas, cada turno renueva el TTL automáticamente; para pipelines batch con intervalos de más de 5 minutos, estructurar el procesamiento para que cada batch comience renovando el caché con una llamada inicial.
- **Google Gemini Context Caching:** API `genai.caching.CachedContent.create()` con TTL configurable y costo de \$1/M tokens/hora; breakeven depende del volumen de llamadas que reutilizan el mismo contenido y del precio diferencial de generación con caché vs sin caché.

El prompt caching es la optimización de costo de mayor impacto para sistemas con system prompts o contextos documentales largos y repetitivos; habilitarlo puede reducir el costo de las llamadas en un 40-80% sin ningún cambio en la calidad de las respuestas, lo que lo convierte en una de las primeras optimizaciones a implementar cuando el sistema alcanza un volumen de tráfico significativo.

Para recordar: El prompt caching es la optimización de costo de mayor ratio impacto/complejidad disponible en el ecosistema de LLMs: unos pocos campos adicionales en el request pueden reducir el costo de los tokens de entrada repetitivos en un 90%, sin ningún cambio en la calidad de las respuestas del sistema.

Módulo 5 – Capítulo 09 – Sección 04

Elección de modelo por costo-beneficio: cuándo usar modelos más pequeños

La elección del modelo de LLM para cada tarea es la decisión de costo-calidad con mayor impacto en el presupuesto total de un sistema de IA: usar `gpt-4o` para clasificar intenciones de usuario con cinco categorías en lugar de `gpt-4o-mini` puede ser quince a veinte veces más caro por request sin ninguna mejora medible en la tarea específica. El principio de routing por complejidad —asignar el modelo más capaz y costoso solo a las tareas que genuinamente lo requieren— puede reducir el costo total del sistema en un 40 a 70% sin degradar la calidad percibida por el usuario.

La forma correcta de decidir si una tarea necesita el modelo grande es empírica: ejecutar la tarea sobre el dataset de evaluación del dominio con modelos de diferentes capacidades, medir las métricas de calidad relevantes para esa tarea, y calcular el diferencial de calidad vs el diferencial de costo. Si `claude-3-haiku` obtiene un accuracy del 94% en clasificación de intenciones mientras `claude-3-5-sonnet` obtiene el 96%, y el costo por request es 15 veces mayor con el modelo grande, el modelo pequeño es la elección correcta: la diferencia de 2 puntos de accuracy no justifica un incremento de costo del 1.400%. Si el modelo pequeño obtiene el 78% mientras el grande obtiene el 95%, la diferencia de 17 puntos puede justificar el costo adicional dependiendo del impacto de los errores de clasificación en el negocio.

Las categorías de tarea con características bien establecidas permiten una orientación general que debe validarse con datos del dominio específico. La clasificación con pocas categorías bien definidas —intención del usuario con cinco a diez categorías, sentimiento con tres valores— es una de las tareas donde los modelos pequeños superan consistentemente el 90% de la calidad del modelo grande con instrucciones y few-shot examples correctos. La extracción de entidades con formato JSON definido —nombre, fecha, importe, categoría— alcanza el 95% o más de accuracy con modelos pequeños en dominios con vocabulario estable. Los resúmenes de documentos largos y el razonamiento multi-paso con múltiples constraints simultáneos son donde el modelo grande típicamente justifica su costo: la diferencia de calidad es de diez a veinte puntos en estos dominios.

El patrón LLM router lleva el principio al extremo operacional: un modelo ligero y rápido — Claude 3 Haiku, gpt-4o-mini— clasifica la complejidad de cada query del usuario y la enruta al modelo adecuado. La clasificación de routing es rápida (50-100ms con un modelo pequeño), barata (\$0.00003 por request a precios de Haiku), y la distribución de rutas se convierte en una métrica operativa que informa el diseño del sistema: si el 85% del tráfico se clasifica como "simple", el sistema tiene espacio para reducir el costo total usando el modelo pequeño para esa mayoría y reservando el modelo grande para el 15% restante de queries complejas.

Criterios técnicos para la selección de modelo por tarea

- **Clasificación e intent detection:** modelos pequeños (Haiku, gpt-4o-mini) son suficientes con buenas instrucciones y 5-10 ejemplos few-shot; la diferencia de accuracy vs el modelo grande es típicamente de 1 a 5 puntos en dominios con categorías bien definidas.
- **Extracción de entidades simples:** nombre, fecha, importe, categoría de texto estructurado → modelos pequeños con output JSON alcanzan el 95%+ de accuracy vs el modelo grande a diez a veinte veces menor costo; la calidad del few-shot prompt impacta más que el modelo.
- **Resumen de documentos largos:** los modelos pequeños tienden a omitir detalles o a repetir información; el modelo grande justifica su costo en resúmenes de alta calidad donde la completitud y la coherencia son requisitos; validar con métricas de BERTScore o coherence en el dataset del dominio.
- **Razonamiento multi-paso y coding:** mantener múltiples constraints simultáneos, razonamiento causal complejo, generación de código correcto → el modelo grande ofrece mejoras de diez a veinte puntos de calidad que justifican el costo adicional en la mayoría de los casos.
- **LLM router pattern:** `small_model.classify(query)` devuelve `{"complexity": "simple|medium|complex", "task_type": "classification|generation|reasoning"}`; enruta a diferentes endpoints con diferentes modelos; loggear el modelo usado para análisis de distribución de rutas y optimización continua.

Nota del Arquitecto: El error más frecuente en la selección de modelo es asumir que "más grande es mejor" sin medir en el dominio específico. En un proyecto de clasificación de tickets de soporte técnico en telecomunicaciones que implementé, Claude 3 Haiku con diez ejemplos few-shot específicos del dominio superó a Claude 3 Sonnet sin few-shot en todas las métricas relevantes, a un costo quince veces menor.

El few-shot prompting específico del dominio importó más que la capacidad del modelo. Este resultado no es generalizable —cada dominio es diferente— pero sí es la razón por la que la medición empírica siempre supera a la intuición sobre capacidad del modelo.

El modelo correcto no es el más capaz disponible sino el menos capaz que cumple el requisito de calidad medido en el dataset de evaluación del dominio específico.

Principio rector: Sobre-aprovisionar modelo para una tarea es tan ineficiente como sub-aprovisionar infraestructura: el modelo correcto no es el más capaz disponible sino el menos capaz que cumple el requisito de calidad medido empíricamente en el dataset de evaluación del dominio específico de la tarea.

Módulo 5 – Capítulo 09 – Sección 05

Monitoreo y alertas de costos: dashboards y umbrales de gasto

El monitoreo de costos en sistemas de IA tiene una característica que lo distingue del monitoreo de costos de infraestructura tradicional: el costo puede cambiar de forma impredecible sin ningún cambio en el código de la aplicación, por factores externos al control del equipo como el crecimiento orgánico del historial conversacional, el aumento de consultas complejas por cambios en el comportamiento del usuario, o bugs que generan loops de llamadas al LLM. Un sistema de monitoreo de costos que solo alerta cuando se alcanza el límite mensual del proveedor es un sistema reactivo; un sistema que detecta anomalías intradía en el patrón de gasto es un sistema que puede responder antes de que el problema escale.

El monitoreo de costos opera efectivamente en tres horizontes temporales con herramientas diferentes. En tiempo real —ventanas de 15 a 60 minutos—, el objetivo es detectar spikes anómalos de gasto que pueden indicar un bug crítico: un loop infinito de llamadas al LLM, un usuario con comportamiento anómalo que está realizando miles de requests, o un prompt que creció inesperadamente por un bug de acumulación de contexto. Las alertas de anomalía —CloudWatch Anomaly Detection, Grafana Alerting con modelo predictivo— son más efectivas que los umbrales estáticos para detectar estos spikes porque se adaptan automáticamente al patrón normal de uso del sistema. En el horizonte diario, el objetivo es el breakdown de costos por feature, modelo y segmento de usuario, que informa las decisiones de optimización. En el horizonte mensual, el objetivo es la tendencia y la proyección que informa el presupuesto y la planificación de capacidad.

Los dashboards de costos en Grafana tienen tres paneles esenciales. El panel "Costo por hora (rolling 24h)" con alerta si supera el percentil 95 histórico detecta spikes en tiempo casi real. El panel "Top features por costo (últimos 7 días)" con breakdown por `feature` y `model` identifica las oportunidades de optimización más grandes: el feature que más cuesta es el primero a evaluar para optimización de modelo o de contexto. El panel "Proyección mensual"

que extrapola el gasto diario promedio del último período al mes completo hace visible la tendencia antes de que el presupuesto se agote, permitiendo ajustes proactivos.

La atribución de costos por equipo o por feature requiere que el sistema de logging tenga los campos correctos desde el inicio. Un campo `team` o `feature` en cada log de llamada al LLM permite queries del tipo "¿cuánto gastó el equipo de producto en el feature de análisis de documentos la última semana?" directamente desde el sistema de logging, sin necesidad de una integración adicional con un data warehouse. Esta granularidad de atribución es la que convierte las reuniones de revisión de presupuesto de debates sobre números totales a conversaciones sobre qué features tienen el mejor ROI de IA.

Aspectos técnicos del monitoreo de costos

- **Cálculo local del costo en cada request:** tabla de precios como constante de configuración (`PRICING = {"claude-3-5-sonnet-20241022": {"input": 3.0, "output": 15.0, "cache_read": 0.30, "cache_write": 3.75}}`); costo calculado en cada respuesta y loggeado como `cost_usd` ; no depender solo de las métricas del dashboard del proveedor para el monitoreo operacional.
- **Alerta de anomalía intradía:** CloudWatch Anomaly Detection o Grafana Alerting con modelo de predicción de gasto basado en el histórico de 30 días; detecta spikes que son estadísticamente anómalos respecto al patrón normal sin requerir definir un umbral estático.
- **Budget alerts del proveedor:** configurar alertas al 50%, 80% y 100% del presupuesto mensual en la consola de OpenAI o Anthropic como safety net de última línea; complementa el monitoreo propio pero no lo reemplaza porque llega tarde y sin granularidad de feature.
- **Atribución por equipo y feature:** campo `feature` en cada log de llamada; queries `SELECT feature, SUM(cost_usd) FROM llm_calls WHERE created_at > NOW() - INTERVAL 7 DAYS GROUP BY feature ORDER BY 2 DESC` identifican los mayores consumidores de costo.
- **Dashboard de proyección mensual:** panel con `total_cost_last_7_days / 7 * 30` como proyección simple; alerta si la proyección supera el 80% del presupuesto con más de diez días de mes restantes, dando tiempo para aplicar optimizaciones antes del cierre del período.

El costo de construir el sistema de monitoreo de costos en las primeras semanas del proyecto es invariablemente menor que el costo de descubrir un bug de loop infinito o un prompt mal

acotado en la primera factura mensual, y órdenes de magnitud menor que el costo de ese mismo bug en un sistema con un año de usuarios y datos de producción.

Buena práctica: El costo de construir el sistema de monitoreo de costos en las primeras semanas del proyecto es invariablemente menor que el costo de descubrir un bug de loop infinito o un prompt sin límite de longitud en la primera factura mensual; el monitoreo de anomalías intradía es el único mecanismo que detecta estos problemas antes de que escalen.

Módulo 5 – Capítulo 09 – Sección 06

Cierre: optimización de costos sin comprometer la calidad del sistema

La optimización de costos en sistemas de IA es un proceso de ingeniería riguroso, no de austeridad ciega. La diferencia entre estas dos perspectivas es la medición: la austeridad ciega reduce costos esperando que la calidad no se degrade significativamente; la ingeniería rigurosa mide el impacto de cada cambio en las métricas de calidad del dataset de evaluación antes de desplegarlo en producción, y solo acepta el cambio si el ahorro justifica la degradación que produce (si produce alguna). Este rigor protege contra el escenario más común de optimización mal ejecutada: ahorrar un 20% en costos reduciendo el contexto RAG de cinco chunks a dos chunks, y descubrir tres semanas después que la tasa de feedback negativo subió un 15% porque las respuestas ahora carecen del contexto necesario para ser precisas.

El proceso correcto de optimización de costos tiene cuatro pasos en secuencia. Primero, medir el costo actual con granularidad suficiente: costo por request desagregado por feature, por tipo de tarea, por segmento de usuario, y por componente (entrada, salida, caché). Sin esta granularidad, no es posible identificar las oportunidades de ahorro más grandes ni cuantificar el impacto de cada cambio. Segundo, identificar los contribuyentes más grandes al costo total: frecuentemente el 20% de los features representa el 80% del gasto, y dentro de esos features, el 20% de los patrones de uso representa el 80% del costo de ese feature. Tercero, implementar la optimización en una rama separada y medir su impacto en el dataset de evaluación antes de cualquier despliegue: si la optimización degrada alguna métrica de calidad, determinar si la degradación es aceptable o no antes de comprometerse. Cuarto, desplegar en canary con monitoreo de métricas de calidad en tiempo real antes del rollout completo.

Las optimizaciones de mayor retorno en la práctica, en orden de ROI típico: la migración de tareas simples —clasificación, extracción, routing— al modelo más pequeño que cumple el requisito de calidad medido en el dataset (30-80% de ahorro); la implementación de prompt caching en system prompts largos y documentos de referencia fijos (40-90% de ahorro en

tokens de entrada repetitivos); la reducción del contexto RAG de cinco chunks a tres cuando el sistema de evaluación confirma que los chunks adicionales no contribuyen a la calidad (33% de ahorro en tokens de contexto); y la implementación de caché semántica para queries repetitivas en dominios estables (20-50% de ahorro según el patrón de uso). Estas cuatro optimizaciones, aplicadas en ese orden de ROI, pueden reducir el costo total del sistema en un 50-70% sin ninguna degradación de calidad perceptible para el usuario.

La revisión trimestral del stack de modelos y precios es una práctica de ingeniería de costo que tiene retorno comprobable: los precios de los proveedores bajan regularmente (OpenAI redujo los precios de GPT-4 Turbo en un 50% en 2024), nuevos modelos más eficientes se lanzan cada trimestre, y el modelo que era la elección óptima hace seis meses puede no serlo hoy. Un equipo que revisa activamente el landscape cada trimestre puede ahorrar del 20 al 40% anual solo por mantenerse actualizado con las opciones disponibles.

Principios de optimización de costos sin degradar calidad

- **Medir antes de optimizar:** establecer el baseline de costo y calidad antes de cualquier cambio; sin baseline, no es posible cuantificar el ahorro real ni la posible degradación de calidad.
- **Optimizar con evidencia empírica del dominio:** cada optimización propuesta debe probarse sobre el dataset de evaluación del dominio específico; generalizaciones de "los modelos pequeños son suficientes para clasificación" no sustituyen la medición en el dominio propio.
- **Mantener slack en el presupuesto:** diseñar el sistema para operar cómodamente al 60-70% del presupuesto mensual; el 30-40% de margen absorbe picos de tráfico, experimentos y el crecimiento natural de la base de usuarios sin crisis de presupuesto.
- **Revisión trimestral del stack de modelos:** los precios bajan y nuevos modelos más eficientes aparecen regularmente; una revisión trimestral puede identificar opciones más económicas con calidad equivalente que el equipo no está usando por inercia.
- **Costo por valor generado:** medir el costo de IA como porcentaje del valor de negocio que el feature genera (revenue, tiempo ahorrado, CSAT mejorado); un feature que cuesta \$5.000/mes pero genera \$50.000 en valor tiene un ROI del 900% que justifica mantenerlo y expandirlo sin presión de reducción de costos.

"The fastest code is the code which does not run." — Mike Haertel, adaptado. En AI Engineering: el token más barato es el token que no se envía; la optimización de costos más efectiva es eliminar los tokens innecesarios del prompt y el contexto antes de explorar

cambios de modelo o configuración. La concisión bien diseñada en los prompts reduce costos, reduce latencia, y frecuentemente mejora la calidad de las respuestas al reducir el ruido en el contexto.